Computer Architecture Mini-Project 2 Report

Introduction

This report details the design and implementation of a digital circuit using the OSS CAD suite to drive an RGB LED on an FPGA board. It uses pulse width modulation (PWM) to smoothly cycle around the HSV color wheel once per second. The circuit was implemented using SystemVerilog and tested with a simulation in Icarus Verilog (iverilog). This report details the circuit design, its operation, simulation results, and provides the source code used in the project.

Circuit Design

The circuit is composed of three main modules described below:

1. **Color_fade.sv:** generates RGB intensity values corresponding to different hues in the HSV color wheel.
    - Implements a hue step counter (`step_counter`) that increments at a rate determined by the system clock to transition through 360 hue steps per second.
    - Divides the hue spectrum into six distinct regions, each corresponding to a different primary or secondary color transition.
    - Within each region, the RGB values are updated using linear interpolation to create smooth color transitions.
    - `Cycle_counter` ensures that each hue step remains for a fixed number of clock cycles before moving to the next step.

2. **Pwm.sv:** generates pulse-width modulated signals based on the input duty cycle values for red, green, and blue channels.
    - A counter (`pwm_count`) continuously increments, comparing its value to the duty cycle input to determine the output state.
    - The PWM signal is high when the counter is less than the duty cycle and low otherwise.
    - The module ensures a high enough PWM frequency to eliminate visible flickering.

3. **Top.sv:** integrates the `color_fade` and `pwm` modules to control the RGB LED
    - The `color_fade` module provides dynamically changing RGB values to the PWM modules.
    - Three instances of the `pwm` module generate independent PWM signals for the red, green, and blue channels.
    - The outputs are inverted (`~pwm_r, ~pwm_g, ~pwm_b`) due to the iceBlinkPico board's RGB LED operating in active-low mode.

Circuit Operation

The circuit operates as follows:

1. The 12 MHz system clock drives the `color_fade` module
2. The `color_fade` module computes the RGB intensity values based on the current hue step.
3. The `pwm` module takes these intensity values and generates PWM signals to drive the LED, adjusting brightness levels accordingly.
4. The RGB LED gradually transitions through the HSV color wheel over a one-second period.
5. The cycle repeats indefinitely, maintaining smooth and continuous color transitions.

Simulation Results

To validate the design, a SystemVerilog testbench (`color_fade_tb.sv`) was created to simulate the circuit using Icarus Verilog. The testbench functions as follows:

- Generates a 12 MHz clock signal.
- Instantiates the top module and monitors its RGB outputs.
- Captures the RGB values over time.
- Stores the simulation results in a VCD (value change dump) file for waveform visualization in GTKWave.

A screenshot of the waveform is provided below. It shows the RGB values smoothly and gradually increasing and decreasing, changing with time. This confirms the expected smooth color cycle behavior.

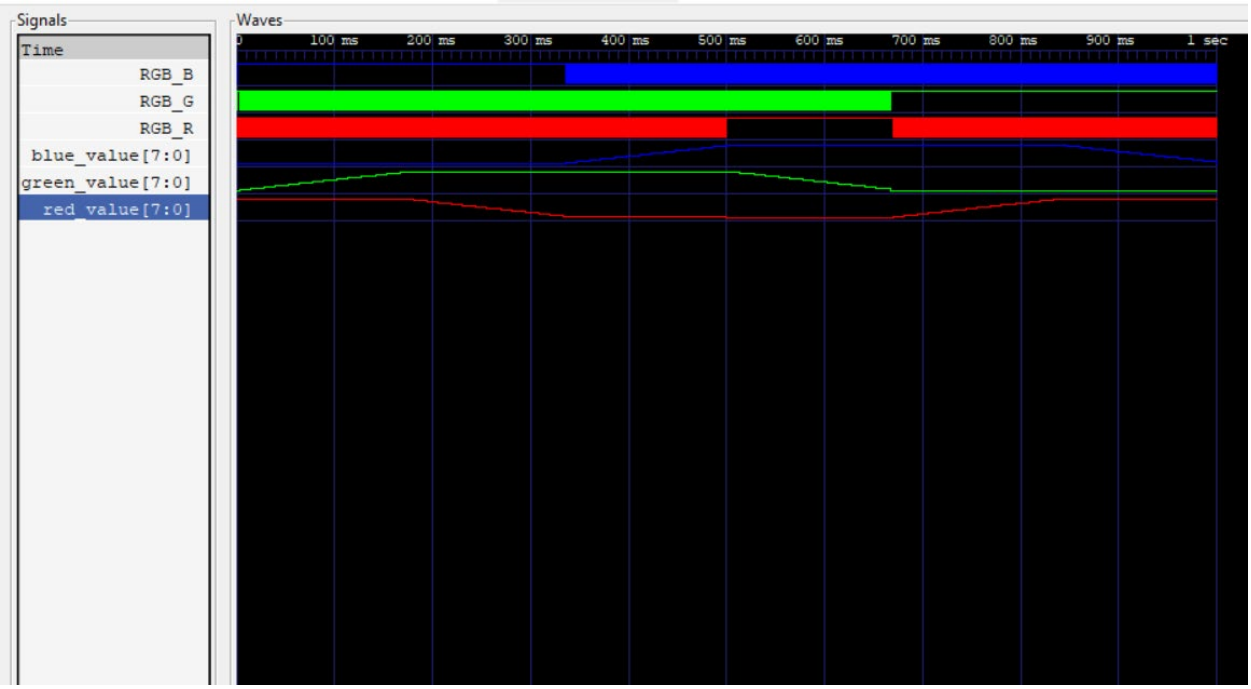

Figure 1: GTKWave simulation results showing RGB signal transitions over time

Source Code

Color_fade_tb.sv

```
`timescale 1ns/1ps
`include "top.sv"

module top_tb;

    logic clk = 0;
    logic RGB_R;
    logic RGB_G;
    logic RGB_B;

    // Instantiate the top module instead of color_fade
    top uut (
        .clk(clk),
        .RGB_R(RGB_R),
        .RGB_G(RGB_G),
        .RGB_B(RGB_B)
    );

    // Generate clock signal
    always begin
        #41.6667 clk = ~clk; // Approximate 24 MHz clock
    end

    initial begin
        clk = 0;
        $dumpfile("top.vcd");
        $dumpvars(0, top_tb);
        #1000000000; // Run long enough to observe PWM transitions
        $finish;
    end

endmodule
```

Color_fade.sv

```
// Fade
// Smoothly cycles through HSV color wheel using PWM

module color_fade #(
    parameter CLK_FREQ = 12_000_000,  // 12 MHz clock
    parameter TOTAL_STEPS = 360,      // Total hue transition steps
    parameter COLOR_INTERVAL = CLK_FREQ / TOTAL_STEPS  // Cycles per hue step
)(
    input logic clk,
    output logic [7:0] red, green, blue
);

    localparam int STEP_SIZE = 60;   // Number of steps per color transition

    logic [$clog2(COLOR_INTERVAL)-1:0] cycle_counter = 0;  // Counts clock cycles
    logic [8:0] step_counter = 0;    // Hue step counter (0-359)
    logic [2:0] hue_region;          // Stores which hue transition led is in

    // RGB values
    logic [7:0] red_value   = 8'd255;
    logic [7:0] green_value = 8'd0;
    logic [7:0] blue_value  = 8'd0;

    // Step counter logic (advances hue every `COLOR_INTERVAL` cycles)
    always_ff @(posedge clk) begin
        if (cycle_counter >= COLOR_INTERVAL - 1) begin
            cycle_counter <= 0;
            if (step_counter >= TOTAL_STEPS - 1)
                step_counter <= 0;
            else
                step_counter <= step_counter + 1;
        end
        else begin
            cycle_counter <= cycle_counter + 1;
        end
    end
```

```
    // Assign hue region using division
    always_comb begin
        hue_region = step_counter / STEP_SIZE;
    end

    // Compute step within the current region
    logic [8:0] step_in_region;
    always_comb begin
        step_in_region = step_counter - (hue_region * STEP_SIZE);
    end

    // Assign RGB values based on hue region (smooth color transitions)
    always_comb begin
        case (hue_region)
            3'd0:  {red_value, green_value, blue_value} = {8'd255, step_in_region * 4'd4, 8'd0};  // Red -> Yellow
            3'd1:  {red_value, green_value, blue_value} = {8'd255 - step_in_region * 4'd4, 8'd255, 8'd0};  // Yellow -> Green
            3'd2:  {red_value, green_value, blue_value} = {8'd0, 8'd255, step_in_region * 4'd4};  // Green -> Cyan
            3'd3:  {red_value, green_value, blue_value} = {8'd0, 8'd255 - step_in_region * 4'd4, 8'd255};  // Cyan -> Blue
            3'd4:  {red_value, green_value, blue_value} = {step_in_region * 4'd4, 8'd0, 8'd255};  // Blue -> Magenta
            3'd5:  {red_value, green_value, blue_value} = {8'd255, 8'd0, 8'd255 - step_in_region * 4'd4};  // Magenta -> Red
            default: {red_value, green_value, blue_value} = {8'd255, 8'd0, 8'd0};  // Default to red
        endcase
    end

    // Assign outputs
    assign red   = red_value;
    assign green = green_value;
    assign blue  = blue_value;

endmodule
```

Pwm.sv

```
// PWM generator to fade LED

module pwm #(
    parameter PWM_INTERVAL = 256  // 8-bit range
)(
    input logic clk,
    input logic [7:0] duty_cycle,
    output logic pwm_out
);

    logic [7:0] pwm_count = 0;

    always_ff @(posedge clk) begin
        pwm_count <= pwm_count + 1;
    end

    assign pwm_out = (pwm_count < duty_cycle) ? 1'b1 : 1'b0;

endmodule
```

Top.sv

```
`include "color_fade.sv"
`include "pwm.sv"

module top (
    input logic clk,
    output logic RGB_R,  // Active-Low Red
    output logic RGB_G,  // Active-Low Green
    output logic RGB_B   // Active-Low Blue
);

    logic [7:0] red, green, blue;
    logic pwm_r, pwm_g, pwm_b;

    // Instantiate color fading module
    color_fade u_color_fade (
        .clk(clk),
        .red(red),
        .green(green),
        .blue(blue)
    );

    // Instantiate PWM modules with scaling
    pwm #(.PWM_INTERVAL(256)) pwm_red (
        .clk(clk),
        .duty_cycle(red),  // Now takes 8-bit input
        .pwm_out(pwm_r)
    );

    pwm #(.PWM_INTERVAL(256)) pwm_green (
        .clk(clk),
        .duty_cycle(green),
        .pwm_out(pwm_g)
    );
```

```
    pwm #(.PWM_INTERVAL(256)) pwm_blue (
        .clk(clk),
        .duty_cycle(blue),
        .pwm_out(pwm_b)
    );

    // Assign RGB outputs (Active Low)
    assign RGB_R = ~pwm_r;
    assign RGB_G = ~pwm_g;
    assign RGB_B = ~pwm_b;

endmodule
```